

DEEP NEURAL NETWORKS: CẨM NANG TOÀN DIỆN

Tổng hợp Nền tảng Lý thuyết Chuyên sâu & Phương pháp Áp dụng Thực tế (Lecture 03)

1. Cấu trúc Nơ-ron & Phép tính Cốt lõi (Neuron Mechanics)

Mạng Nơ-ron Nhân tạo (Artificial Neural Network - ANN) là một mô hình học máy lấy cảm hứng từ não bộ sinh học. Thành phần cơ bản nhất của mạng là các **Nơ-ron (Neurons/Nodes)** được liên kết với nhau theo từng lớp. Mỗi nơ-ron nhận dữ liệu đầu vào, thực hiện hai phép tính liên tiếp và truyền kết quả cho lớp tiếp theo.

Hai công đoạn tính toán tại từng nơ-ron:

- Công đoạn 1: Tính Tổng có trọng số (Weighted Sum - z)**

Tín hiệu đầu vào x được nhân với các trọng số w tương ứng và cộng thêm độ lệch (bias b):

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \mathbf{w}^T\mathbf{x} + b$$

- Trọng số (w - Weights): Đo lường mức độ quan trọng/ảnh hưởng của từng tín hiệu đầu vào. Trọng số do **máy tự học và tìm ra** trong quá trình huấn luyện.
- Độ lệch (b - Bias): Hằng số dịch chuyển ngưỡng kích hoạt, giúp mô hình linh hoạt hơn khi đầu vào $x = 0$.

- Công đoạn 2: Biến đổi qua Hàm kích hoạt (Activation Function - $a = g(z)$)**

Giá trị z được đưa qua hàm kích hoạt $g(z)$ để tạo ra đầu ra chính thức $a = g(z)$.

- Vai trò bắt buộc của $g(z)$ - Tính Phi tuyến (Nonlinearity):** Nếu không có $g(z)$, toàn bộ mạng nơ-ron dù có hàng trăm lớp cũng chỉ tương đương một phương trình tuyến tính đơn giản (đường thẳng/mặt phẳng). Hàm kích hoạt uốn cong ranh giới quyết định, giúp mạng học được các mối quan hệ phi tuyến phức tạp.

ẢNH DỤ THỰC TẾ: NHÀ BẾP NẤU BÁNH

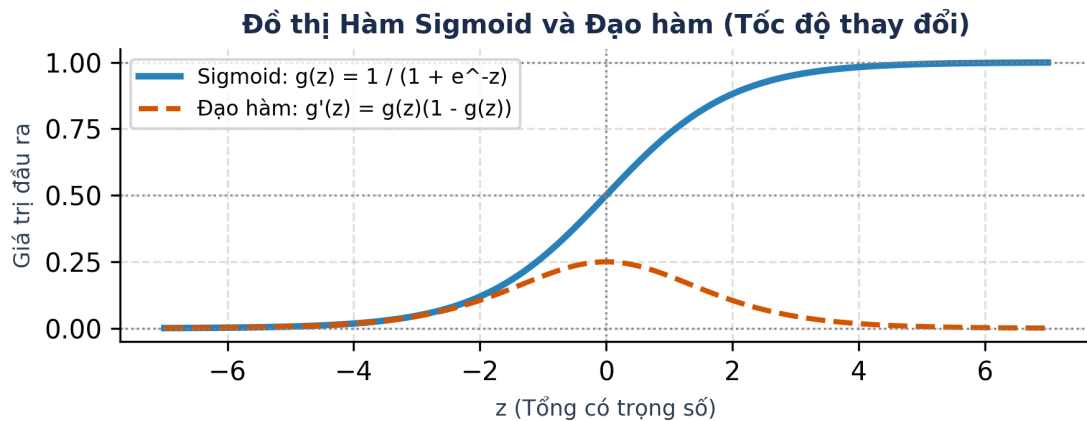
- Nơ-ron** = Người thợ làm bánh.
- Trọng số w & Bias b** = Công thức nêm gia vị (2 thìa đường, 3 thìa bột, 1 chút muối). Ban đầu chưa biết công thức, thợ nêm ngẫu nhiên.
- Hàm kích hoạt $g(z)$** = Chiếc lò nướng biến hỗn hợp bột thô (z) thành chiếc bánh hoàn chỉnh có hình dáng đẹp (a).

2. Hàm Kích hoạt: Sigmoid & ReLU

Lecture 03 nhấn mạnh hai hàm kích hoạt nền tảng nhất trong Deep Learning là **Sigmoid** và **ReLU**. Mỗi hàm có vai trò và vị trí áp dụng riêng biệt.

Tiêu chí	Hàm Sigmoid: $g(z) = 1 / (1 + e^{-z})$	Hàm ReLU: $g(z) = \max(0, z)$
Miền giá trị	Khoảng $(0, 1)$ mượt mà	$[0, +\infty)$
Đạo hàm (Derivative)	$g'(z) = g(z) \cdot (1 - g(z))$	1 nếu $z > 0$; 0 nếu $z < 0$

Tiêu chí	Hàm Sigmoid: $g(z) = 1 / (1 + e^{-z})$	Hàm ReLU: $g(z) = \max(0, z)$
Ưu điểm chính	Đầu ra dạng xác suất, liên tục và mượt	Tính toán siêu nhanh, không bị triệt tiêu gradient ở $z > 0$
Nhược điểm chí mạng	Triệt tiêu Gradient (Vanishing Gradient) ở z quá lớn/nhỏ	Dying ReLU (nơ-ron ngừng học nếu $z < 0$)
Vị trí áp dụng chuẩn	Lớp Đầu ra (Output Layer) cho Phân loại Nhị phân	Lớp Ẩn (Hidden Layers) - Chuẩn công nghiệp



Hình 1: Đồ thị hàm Sigmoid nén giá trị về (0,1) và đồ thị đạo hàm đạt cực đại 0.25 tại $z=0$.

3. Quy trình Huấn luyện & Hai chiều Lan truyền (Forward & Backward)

Quá trình huấn luyện Mạng Nơ-ron là một vòng lặp gồm 3 giai đoạn nối tiếp nhau nhằm giảm thiểu sai số dự đoán:

- 1. Forward Propagation (Lan truyền tiến - Chiều Trái → Phải):**
Dữ liệu x đi từ Input Layer qua các Hidden Layers đến Output Layer để tạo ra kết quả dự đoán $\hat{y}$. Sau đó, mô hình tính toán **Hàm Mất mát (Loss Function - L)** để đo lường độ sai lệch giữa dự đoán $\hat{y}$ và thực tế y .
 - Hồi quy (Regression): Dùng Mean Squared Error (MSE): $L = (y - \hat{y})^2$
 - Phân loại Nhị phân (Binary Classification): Dùng Binary Cross-Entropy Loss: $L = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$
- 2. Backpropagation (Lan truyền ngược - Chiều Phải → Trái):**
Để sửa lỗi, mô hình chạy ngược từ Output về Input. Thuật toán áp dụng **Quy tắc chuỗi (Chain Rule)** của phép tính đạo hàm để tính **Gradient ($\partial L / \partial w$)** cho từng trọng số w trong toàn bộ mạng. Đạo hàm này cho biết mức độ đóng góp sai số của từng trọng số.
- 3. Weight Update via Gradient Descent (Cập nhật Trọng số):**
Các trọng số được điều chỉnh theo hướng ngược chiều Gradient để hạ thấp mức độ sai số.

BẢN CHẤT TOÁN HỌC CỦA BACKPROPAGATION (CHAIN RULE)

Đạo hàm sai số L theo trọng số w_i tại lớp ẩn được tính bằng tích các đạo hàm từng bước:

$$\partial L / \partial w_i = (\partial L / \partial \hat{y}) \times (\partial \hat{y} / \partial a) \times (\partial a / \partial z) \times (\partial z / \partial w_i)$$

Nhờ tính toán đi ngược từ phải sang trái, kết quả đạo hàm ở lớp sau được tái sử dụng cho lớp trước, giúp tiết kiệm tối đa chi phí tính toán.

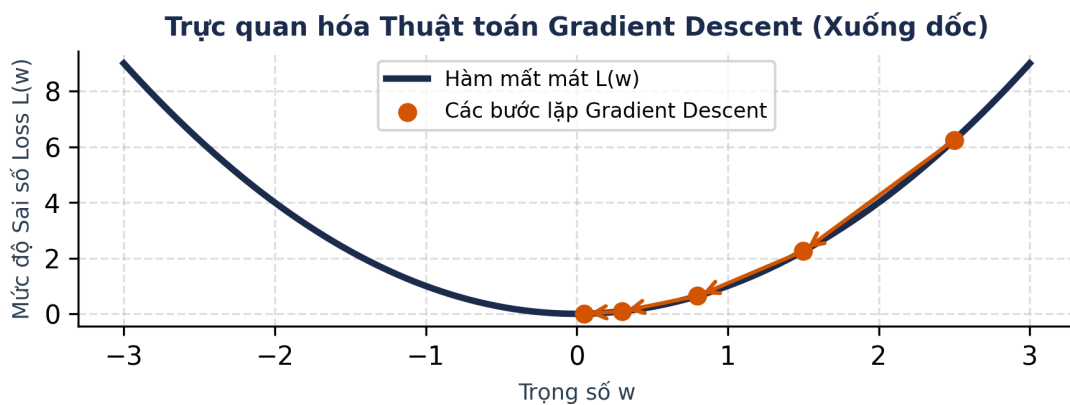
4. Thuật toán Gradient Descent (Xuống dốc)

Gradient Descent là thuật toán tối ưu hóa giúp mô hình tự dò tìm bộ trọng số w sao cho sai số Loss đạt mức nhỏ nhất (Global Minimum).

Công thức cập nhật trọng số:

$$w_{\text{mới}} = w_{\text{cũ}} - \alpha \cdot (\partial L / \partial w)$$

- Dấu **dấu trừ (-)** thể hiện việc di chuyển ngược chiều gradient (đi xuống dốc).
- **α (Alpha / Learning Rate - Tốc độ học):** Đóng vai trò là độ dài bước chân.
 - Nếu α quá nhỏ: Mô hình di chuyển rùa bò, mất rất nhiều thời gian mới hội tụ.
 - Nếu α quá lớn: Bước nhảy quá trớn, mô hình bị phân kỳ (diverge) và bùng nổ sai số.



Hình 2: Mô phỏng quá trình lặp của Gradient Descent đưa trọng số w tiến về đáy sai số nhỏ nhất.

So sánh 3 biến thể của Gradient Descent:

Biến thể	Cơ chế gom dữ liệu	Ưu điểm	Nhược điểm
Batch Gradient Descent	Gom toàn bộ dữ liệu trong tập train cho 1 lần bước	Đường đi cực kỳ mượt và ổn định	Rất chậm và tràn RAM khi dữ liệu lớn
Stochastic GD (SGD)	Lấy ngẫu nhiên 1 mẫu dữ liệu duy nhất cho 1 lần bước	Tính toán siêu nhanh	Đường đi dẫn xóc (zigzag), dễ bị nhiễu
Mini-Batch GD (Chuẩn)	Chia dữ liệu thành các nhóm nhỏ (32, 64, 128 mẫu)	Tận dụng GPU song song, hội tụ nhanh & mượt	Cần cấu hình thêm kích thước batch_size

5. Cẩm nang Áp dụng Thực tế (Practical Application Guide)

Khi thiết kế kiến trúc Mạng Nơ-ron cho bài toán thực tế, hãy tuân theo quy tắc phối hợp Hàm kích hoạt & Hàm mất mát chuẩn sau:

Dạng Bài toán (Task)	Lớp Ẩn (Hidden Layers)	Lớp Đầu ra (Output Layer)	Hàm Mất mát (Loss Function)
Hồi quy (Regression) Ví dụ: Dự đoán giá nhà, nhiệt độ	ReLU	Tuyến tính (Linear / None)	Mean Squared Error (MSE)
Phân loại Nhị phân (Binary) Ví dụ: Spam / Not Spam	ReLU	Sigmoid (Ra xác suất 0-1)	Binary Cross-Entropy
Phân loại Nhiều lớp (Multiclass) Ví dụ: Nhận diện ảnh 0-9	ReLU	Softmax (Tổng xác suất các lớp = 1)	Categorical Cross-Entropy

TÓM TẮT DỄ NHỚ VỚI 4 TỪ KHÓA CỐT LÕI

- Forward Propagation** = Nấu bánh & Đoán kết quả.
- Loss Function** = Giám khảo chấm điểm mức độ đoán sai.
- Backpropagation (Chain Rule)** = Truy cứu trách nhiệm xem trọng số w nào làm sai nhiều nhất.
- Gradient Descent** = Chỉnh sửa trọng số w một bước nhỏ để lần sau đoán chuẩn hơn.